

The Process

CS 232 Operating Systems

Fall 2020

What is a process?

- One of the main jobs of the OS is to run other programs.
- When you create a program it's just a file, like any other, on the file system.
- This file's content is in binary and is executable.
- You can run this program
- A running program is called a Process, a fundamental OS abstraction.
- The same executable program run twice would result in two different processes.

What is a process?

- A process has:
 - a unique Process Identifier (PID)
 - an address space with memory image:
 - . Static (code + data)
 - . Dynamic (stack + heap)
 - a context
 - . Program Counter
 - . Stack Pointer
 - . Frame Pointer
 - . Other values being used
 - resources
 - . File descriptors
 - . CPU Time

Process creation

The OS will:

- Allocate memory
- Create a memory image
 - . Load code, data from disk
 - . Create runtime stack and heap
- Open files
 - . stdout, stdin, stderr by default
- Initialize the context
 - . PC, SP, FP, etc

... and step back.

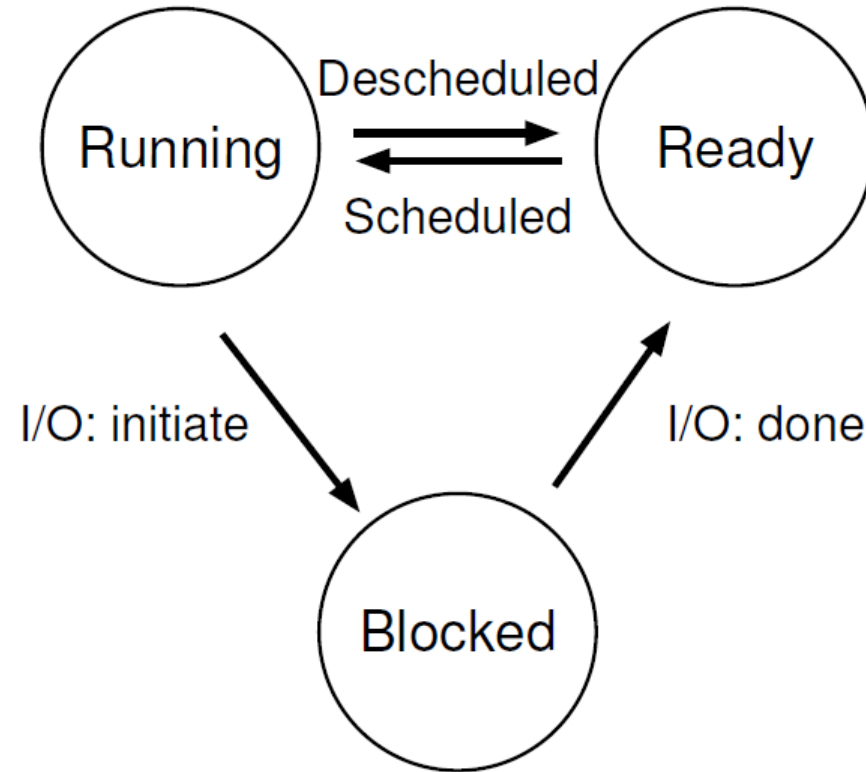
Process states

- Running
- Ready
- Blocked
 - When?

Some OS add other states:

- New
- Dead

Process state transitions



Process state transitions

Time	Process ₀	Process ₁
1	Running	Ready
2	Running	Ready
3	Running	Ready
4	Running	Ready
5	–	Running
6	–	Running
7	–	Running
8	–	Running

Process implementation

- OS maintains a list of processes: ready, blocked, etc
- Data structures: PCB, context, etc
 - Process identifier
 - Process state
 - Process context
 - Process resources
 - Process parent, children info

Process: data structures

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;           // Start of process memory
    uint sz;            // Size of process memory
    char *kstack;        // Bottom of kernel stack
                        // for this process

    enum proc_state state; // Process state
    int pid;              // Process ID
    struct proc *parent;  // Parent process
    void *chan;           // If non-zero, sleeping on chan
    int killed;           // If non-zero, have been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;     // Current directory
    struct context context; // Switch here to run process
    struct trapframe *tf;  // Trap frame for the
                        // current interrupt
};
```

Process: data structures

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```